

■ FlexiPlay

Transferencia de musica entre plataformas de streaming

Segunda Entrega — Mi Proyecto Agil

Suarez Foti, Santiago

Abril 2026

■ 1. Requerimientos del Proyecto

Tipo de relevamiento utilizado: Para identificar los requerimientos se utilizaron entrevistas informales con potenciales usuarios finales (oyentes de musica en multiples plataformas), observacion directa del problema (experiencia propia al compartir musica) y benchmarking de herramientas existentes como Soundiiz y TuneMyMusic.

Requerimientos Funcionales

ID	Descripcion
RF-01	El sistema debe permitir ingresar la URL de una cancion o playlist de YouTube Music, Spotify, Apple Music o Deezer.
RF-02	El sistema debe identificar automaticamente el artista, titulo, album y duracion de la cancion ingresada.
RF-03	El sistema debe generar un link unico y compartible que redirija al usuario a la misma cancion en la plataforma de destino elegida.
RF-04	El sistema debe detectar automaticamente la plataforma de streaming instalada en el dispositivo del receptor del link.
RF-05	El sistema debe mostrar el progreso de conversion en tiempo real al procesar playlists.
RF-06	El sistema debe generar un reporte de canciones no encontradas al finalizar la conversion de una playlist.
RF-07	El sistema debe permitir al usuario registrarse e iniciar sesion con su cuenta de Google.

RF-08	El sistema debe almacenar un historial de las ultimas 50 conversiones realizadas por el usuario registrado.
RF-09	El sistema debe ofrecer un plan gratuito con limite de 5 conversiones mensuales y un plan premium ilimitado.
RF-10	El sistema debe procesar el pago del plan premium a traves de Stripe.

Requerimientos No Funcionales

ID	Categoria	Descripcion
RNF-01	Rendimiento	El tiempo de respuesta para la conversion de una cancion no debe superar los 3 segundos.
RNF-02	Disponibilidad	El sistema debe tener una disponibilidad del 99.5% mensual (menos de 4 horas de caida al mes).
RNF-03	Escalabilidad	La arquitectura debe soportar hasta 10.000 usuarios concurrentes sin degradacion del servicio.
RNF-04	Seguridad	Los datos de usuarios y credenciales OAuth deben almacenarse cifrados. El sistema debe cumplir con GDPR basico.
RNF-05	Usabilidad	Un usuario sin experiencia previa debe poder realizar su primera conversion en menos de 60 segundos.
RNF-06	Compatibilidad	La aplicacion debe funcionar correctamente en los navegadores Chrome, Firefox y Safari, y en dispositivos moviles iOS y Android.
RNF-07	Mantenibilidad	El codigo debe estar documentado y estructurado en modulos independientes por plataforma para facilitar la adicion de nuevas integraciones.

■ 2. Historias de Usuario

Formato: **Como [rol], quiero [accion], para [beneficio]**. Prioridad segun MoSCoW (Must / Should / Could / Won't Have).

HU-01

Epica: **Busqueda**

Must Have

Como usuario, quiero pegar el link de una cancion de YouTube Music, para que FlexiPlay identifique la cancion automaticamente.

Criterios de aceptacion:

- El sistema detecta artista, titulo y album.
- Muestra confirmacion antes de generar el link.
- Tiempo de respuesta menor a 3 seg.

HU-02

Epica: **Conversion**

Must Have

Como usuario, quiero seleccionar Spotify como destino, para recibir el link directo a esa misma cancion.

Criterios de aceptacion:

- El link redirige a la version correcta en Spotify.
- Si no existe la version exacta, muestra la mas similar.
- Notifica si la cancion no esta disponible.

HU-03

Epica: **Playlists**

Must Have

Como usuario, quiero convertir una playlist entera de YouTube Music a Spotify, para no perder mis canciones al cambiar de plataforma.

Criterios de aceptacion:

- Procesa hasta 200 canciones por playlist.
- Muestra progreso en tiempo real.
- Genera reporte de canciones no encontradas.

HU-04

Epica: **Compartir**

Must Have

Como usuario, quiero compartir un link unico de FlexiPlay, para que mis amigos accedan desde su plataforma preferida.

Criterios de aceptacion:

- El link es accesible sin registro.
- Detecta la plataforma del dispositivo receptor.
- Link valido por 30 dias.

HU-05

Epica: **Cuentas**

Have

Como usuario, quiero registrarme con Google, para guardar mis conversiones anteriores.

Criterios de aceptacion:

- Login con Google OAuth 2.0.
- Historial de hasta 50 conversiones.
- Datos seguros y privados.

HU-06

Epica: **Multi-plataforma**

have

Como usuario, quiero convertir entre Apple Music y Deezer, para tener mas opciones de destino.

Criterios de aceptacion:

- Soporte de Apple Music API.
- Soporte de Deezer API.
- Misma experiencia que con Spotify.

HU-07

Epica: **Premium**

have

Como usuario premium, quiero playlists ilimitadas, para gestionar toda mi biblioteca.

Criterios de aceptacion:

- Plan gratuito: 5 conv./mes.
- Plan premium: ilimitado.
- Pago via Stripe.

HU-08

Epica: **Extension**

could have

Como usuario, quiero una extension de Chrome, para convertir sin salir de la plataforma.

Criterios de aceptacion:

- Disponible en Chrome Web Store.
- Boton visible en YT Music/Spotify.
- Conversion en 1 click.

HU-09

Epica: **Estadísticas**

could have

Como usuario premium, quiero ver estadísticas de mis conversiones, para conocer mis generos mas compartidos.

Criterios de aceptacion:

- Top 10 canciones convertidas.
- Grafico por genero.
- Exportar en CSV.

HU-10

Epica: **API Publica**

won't have

Como desarrollador, quiero acceder a la API de FlexiPlay, para integrarla en mis propias apps.

Criterios de aceptacion:

- Documentacion en Swagger.
- Rate limit: 100 req/dia plan gratuito.
- API Key unica por desarrollador.

■ 3. Product Backlog y Sprint Backlog

El **Product Backlog** contiene todas las tareas del proyecto organizadas por sprint. El **Sprint Backlog** seleccionado para el primer ciclo de trabajo corresponde al Sprint 1, que representa las tareas necesarias para construir el nucleo del MVP.

SPRINT BACKLOG — Sprint 1 (semanas 1-2) | Objetivo: construir el nucleo del MVP

ID	Tarea	Responsable	HU
T-01	Configurar repositorio GitHub y entorno CI/CD	Backend Dev	HU-01
T-02	Integrar API de YouTube Music (OAuth + busqueda)	Backend Dev	HU-01
T-03	Integrar API de Spotify (OAuth + busqueda + playlist)	Backend Dev	HU-02
T-04	Algoritmo de matching de canciones (titulo + artista + duracion)	Backend Dev	HU-02,03
T-05	Endpoint REST: POST /convert	Backend Dev	HU-02
T-06	Diseño de wireframes en Figma (Home + Conversion + Resultado)	Frontend Dev	HU-01,02
T-07	Implementar UI Home y pantalla de conversion (React)	Frontend Dev	HU-01,02
T-08	Testing unitario de endpoints de conversion	QA	HU-01,02

Product Backlog completo — vista por sprint:

Sprint 1

T-01 HU-01
Configurar repo GitHub y...

T-02 HU-01
Integrar API YouTube Mus...

T-03 HU-02
Integrar API Spotify

T-04 HU-02,03
Algoritmo matching canci...

T-05 HU-02
Endpoint REST /convert

T-06 HU-01,02
Wireframes Figma

T-07 HU-01,02
UI Home + pantalla conve...

T-08 HU-01,02
Testing unitario endpoi...

Sprint 2

T-09 HU-03
Endpoint /convert-playli...

T-10 HU-03
Barra progreso WebSocket...

T-11 HU-04
Generacion link unico sh...

T-12 HU-04
Deteccion plataforma rec...

T-13 HU-04
Pantalla resultado + sha...

T-14 HU-03
Pantalla progreso playli...

T-15 HU-03,04
Testing E2E playlists Cy...

T-16 Infra
Deploy produccion Railwa...

Sprint 3

T-17 HU-05
Google OAuth 2.0

T-18 HU-05
BD usuarios + historial

T-19 HU-05
Pantalla Mi Historial

T-20 HU-06
Integrar API Apple Music

T-21 HU-06
Integrar API Deezer

T-22 HU-06
Selector plataforma ampl...

T-23 HU-05
Testing auth e historial

T-24 HU-06
Testing Apple + Deezer

Sprint 4

T-25 HU-07
Integracion Stripe premi...

T-26 HU-07
Rate limiting plan gratu...

T-27 HU-07
Pagina pricing + suscrip...

T-28 Infra
Cache Redis canciones po...

T-29 UX
Mejoras UX post-feedback

T-30 HU-07
Testing pago premium E2E

T-31 Docs
Documentacion FAQ y How-...

T-32 Launch
Lanzamiento beta publica

■ 4. Producto Mínimo Viable (MVP)

El MVP valida la hipótesis central: **los usuarios quieren compartir música entre plataformas de forma rápida y simple, sin fricciones**. Se completa al finalizar el Sprint 2 (semana 4).

INCLUIDO	EXCLUIDO
+ Conversión canción/playlist YouTube Music -> Spotify + Generación de link único compartible + Interfaz web minimalista (sin registro obligatorio) + Detección de plataforma del dispositivo receptor + Reporte de canciones no encontradas	- Apple Music y Deezer (Sprint 3) - Login y historial de conversiones - Extensión de Chrome - Plan premium y pagos con Stripe - Dashboard de estadísticas y API pública

Criterio de éxito del MVP: Al finalizar el Sprint 2 (semana 4), al menos 20 usuarios reales pueden convertir una canción de YouTube Music a Spotify en menos de 10 segundos, con satisfacción igual o mayor a 4/5.

■ 5. Hitos del Proyecto



MVP — Lanzamiento

Semana 4 — Conversión YouTube Music -> Spotify funcional. Link compartible en producción. Validación con 20 usuarios reales.

Hito 1 — Expansión Multi-plataforma

Semana 6 — Apple Music y Deezer integrados. Login con Google activo. Historial de conversiones. 50 usuarios activos. Tasa de éxito mayor al 90%.

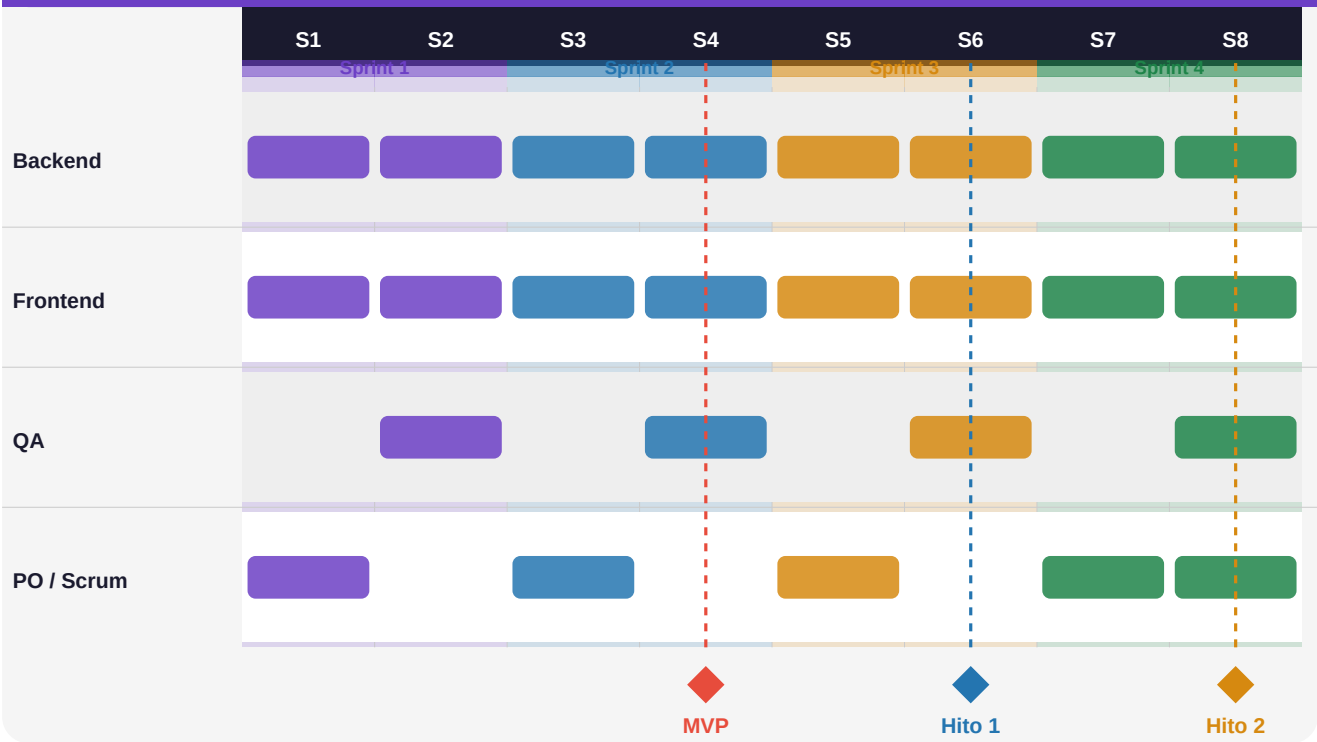
Hito 2 — Monetización y Beta Pública

Semana 8 — Plan premium con Stripe. Rate limiting activo. Beta pública abierta. 200 usuarios registrados. NPS mayor a 40.

6. Roadmap Visual por Semana

Las barras representan la participación de cada rol por semana. Las líneas punteadas indican los hitos clave. Herramienta de construcción: **ReportLab** (Python) — graficos vectoriales embebidos en el PDF.

FlexiPlay — Roadmap Visual | 8 semanas | 4 sprints



Legend: Sprint 1 (purple), Sprint 2 (blue), Sprint 3 (orange), Sprint 4 (green), MVP (red diamond), Hito 1 (blue diamond), Hito 2 (orange diamond).